

Reviewer Pack — Minimal Rank of Tropicalized Quivers Bounds Circuit Monotoni...

Entry d9ef369c3fb2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 21:54:56 UTC

Minimal Rank of Tropicalized Quivers Bounds Circuit Monotonicity

Entry ID: d9ef369c3fb2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 21:54:56 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Quivers) **Field B** (complexity object): Complexity Theory (Boolean Circuit Monotonicity)

Statement:

[‘Let Q be a quiver and let $T(Q)$ be its tropicalization. For any Boolean circuit C with monotone gates, the minimal rank of $T(Q)$ is upper-bounded by the monotonicity degree of C .’, ‘Equivalently, for all monotone Boolean circuits C computing a function $f: \{0,1\}^n \rightarrow \{0,1\}$, there exists a quiver Q such that the tropicalization $T(Q)$ has minimal rank at most equal to the monotonicity degree of C .’, ‘If there exists a Boolean circuit C with monotonicity degree d and a quiver Q such that the minimal rank of $T(Q)$ is greater than d , then C cannot compute its output monotone.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a framework to study discrete structures in a way that can be related to algebraic geometry. Quivers are combinatorial objects that have been studied in tropical geometry and could potentially reveal new insights into the structure of Boolean circuits.’, ‘The minimal rank of a quiver is a measure of its complexity, and bounding this by the monotonicity degree of a circuit would imply a relationship between the combinatorial properties of quivers and the computational complexity of circuits.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6252b549e2dd576c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated circuits with known monotonicity degree d , the minimal rank of $T(Q)$ is at most d in at least 80% of the seeds and the mean minimal rank across seeds does not exceed $d + 1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND quivers AND complexity theory AND boolean circuit monotonicity - minimal rank tropicalization quivers OR quiver tropicalization AND monotonicity degree Boolean circuits - Boolean circuit monotonicity AND upper bound minimal rank tropicalization quiver

Top relevant hits considered: - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1204.3875v2>] Tropicalizing vs Compactifying the Torelli morphism - [<http://arxiv.org/abs/1204.6154v2>] Local Tropicalization - [<http://arxiv.org/abs/1406.3065v2>] Lower Bounds for Tropical Circuits and Dynamic Programs - [<http://arxiv.org/abs/2004.02110v4>] On the cohomology of quiver Grassmannians for acyclic quivers - [<http://arxiv.org/abs/2508.15022v2>] Mutations of quivers with 2-cycles - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, k = len(A), len(B[0])
    n = len(B)
    C = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(n):
                C[i][j] += A[i][l] * B[l][j]
    return C
```

```

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for k in range(i + 1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for k in range(i + 1, n):
            factor = A[k][i] / A[i][i]
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]
            b[k] -= factor * b[i]
    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
    return x

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_circuit(n):
        circuit = []
        for _ in range(2 ** (n - 1)):
            gate_type = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(n)]
            circuit.append((gate_type, inputs))
        return circuit

    def evaluate_circuit(circuit):
        stack = []
        for gate_type, inputs in reversed(circuit):
            if gate_type == 'AND':
                result = all(stack.pop() for _ in range(len(inputs)))
            elif gate_type == 'OR':
                result = any(stack.pop() for _ in range(len(inputs)))
            stack.append(result)
        return stack[0]

    def tropicalize_quiver(circuit):
        n = len(circuit[0][1])
        quiver = [[0] * n for _ in range(n)]
        for gate_type, inputs in circuit:
            if gate_type == 'AND':
                for i in range(n):
                    if inputs[i]:
                        quiver[i][i] += 1
            else:
                quiver[i][i] -= 1
            elif gate_type == 'OR':
                for i in range(n):
                    if inputs[i]:
                        quiver[i][i] += 1
        return quiver

```

```

def minimal_rank(quiver):
    n = len(quiver)
    A = [[quiver[i][j] for j in range(n)] for i in range(n)]
    b = [0] * n
    try:
        x = gaussian_elimination(A, b)
        rank = sum(1 for val in x if abs(val) > 1e-9)
        return rank
    except Exception as e:
        return float('inf')

def monotonicity_degree(circuit):
    n = len(circuit[0][1])
    degree = 0
    for gate_type, inputs in circuit:
        if gate_type == 'AND':
            degree += sum(1 for i in range(n) if inputs[i])
        elif gate_type == 'OR':
            degree += sum(1 for i in range(n) if not inputs[i])
    return degree

n = random.randint(5, 40)
circuit = generate_boolean_circuit(n)
monotonicity = monotonicity_degree(circuit)
quiver = tropicalize_quiver(circuit)
rank = minimal_rank(quiver)

return {
    "metric_name": "minimal_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank <= monotonicity,
    "counterexample": "" if rank <= monotonicity else f"Monotonicity degree {monotonicity}, but minimal rank is {rank}"
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(res["conjecture_holds"] for res in results):
        mean_rank = sum(res["metric_value"] for res in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        counterexample_desc = next(res["counterexample"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={counterexample_desc} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify if the conjecture is supported or falsified according to the pre-registered support condition. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12717
2	preregistration	ollama_remote	glm4:latest	0	0	5467
3	novelty	ollama_remote	glm4:latest	0	0	4813
4	novelty	ollama_remote	glm4:latest	0	0	6534
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21910
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9287
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9133
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18468
9	judge	ollama_remote	glm4:latest	0	0	21893

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110223 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d9ef369c3fb2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d9ef369c3fb2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d9ef369c3fb2.tar.gz` (if generated)